

Project 1 — The Grader

CS496 AI Engineering

Dr Abdeldjalil Labeled

Mediterranean Institute of Technology

Fall 2026



Outline

The brief

The task

The harness and the judge

What to measure

What you hand in

The brief

Build an **evaluation harness** and a **prompted system** for a task you choose. **The harness is the deliverable.** The system exists only to be measured. You will reuse this harness for the rest of the term, so build it well.

You hand in: a golden dataset (150+ items) with a labelling guide, a harness that gives a result for every item, an LLM judge with a reliability report, and prompts under version control with a changelog.

- **Time:** Weeks 2–4
- **Weight:** 15% of the final grade
- **Team:** max 4 students, same team all term
- **Deadline:** repo frozen before the first session of Week 5
- **Demo:** first session of Week 5. Someone will try to break your judge.
- **Grading focus:** Measured 40%, Works 25%, Documented 25%

Objective

Project 0 used tasks with one correct answer. Real tasks often do not have one. This project builds the tool that lets you **measure quality when there is no exact answer.**

At the end of the project, every team member can:

1. **Build a golden set:** 150+ items you labelled yourselves, with a written guide so others label the same way.
2. **Write a harness:** run any system on the set and get one result per item, not just one number.
3. **Build an LLM judge:** a model that grades answers, and proof that it agrees with humans.
4. **Find judge bias:** show whether the judge prefers the first answer, or the longer answer.
5. **Improve with numbers:** every prompt change is linked to a change in the eval score.
6. **Use structured output:** make the model answer in a fixed schema so the harness can read it.

A good task: say yes to all five

1. **There is no single correct answer**, but a careful person can say if an answer is good or bad.
2. **You can write a labelling guide**. The rules for “good” fit on two pages.
3. **You can make 150+ items in one week**, and two people label each one.
4. **One model call per item**. No agents, no retrieval.
5. **You will want this task again**. You will reuse the harness in Projects 2, 3 and 5.

If exact match works, the task is too easy for this project. If two humans cannot agree, it is too vague.

Task ideas

Pick one, or propose your own. Get the instructor's OK by the end of Week 2.

- **Customer support replies**
is the reply correct, polite, and complete?
- **Meeting notes to action items**
are all actions found, with the right owner?
- **Code review comments**
is the comment correct and useful?
- **Feedback on student essays**
does the feedback match a rubric?
- **Product descriptions from specs**
accurate, no invented features, right tone?
- **Emails written to a style guide**
does the email follow every rule in the guide?
- **FAQ answers from a policy document**
correct, complete, nothing made up?
- **Explaining a SQL query in plain words**
is the explanation right and clear?
- **Study notes from a lecture transcript**
are the key points there and correct?
- **Translation quality (Arabic / French / English)**
meaning kept, natural wording?

The golden set and the harness

The golden set (150+ items)

- Two people label every item. Measure how often they agree and report it.
- A **labelling guide**: what “good” means, with examples of good and bad answers.
- Split: a dev set to improve the prompt, a test set you only use to report results.
- Stored as data (`jsonl`), committed to the repo.

The harness

- Runs any system on the whole set with one command.
- Writes **one row per item**: input, output, score, reason.
- Asks the model for **structured output** (a fixed JSON schema) so scoring never depends on text parsing.
- Prompts live in files under git. A **changelog** links every change to the score before and after.

The LLM judge

What it does

- A model reads the input and the answer, and gives a grade using your labelling guide.
- It must return a grade **and a short reason** in a fixed schema.
- It is cheaper than humans, but only useful if it agrees with them.

The reliability report

- **Agreement with humans:** run the judge on the human-labelled items and report how often it agrees.
- **Position bias:** when comparing two answers, swap the order. Does the verdict change?
- **Verbosity bias:** make an answer longer without making it better. Does the grade go up?
- Say clearly where the judge fails. Demo day will test this.

A judge you have not tested is just another model with an opinion.

What to measure

Labels

- Agreement between your two labellers
- How many items were dropped, and why

Judge

- Agreement between judge and humans
- Change of verdict after swapping order
- Change of grade after padding the answer
- Cost and latency per judged item

System

- Score on the test set, per item and overall
- Score after each prompt change (changelog)
- Cost per 1k requests, today and at 100×

Report numbers as counts and percentages. Never only one summary number.

What you hand in

The repo

- `README.md`: one setup command, one run command, results, a Contributions section (one entry per member)
- `data/`: golden set + labelling guide
- `harness/`: runner, scorer, judge, schema
- `prompts/`: versioned prompts + `CHANGELOG.md`
- `results/`: per-item results for every run
- `report.pdf`, `postmortem.md`
- No API keys in the repo

The report (max 4 pages)

1. The task and why exact match is not enough
2. The golden set: how you built and labelled it, agreement numbers
3. The harness: how it runs and what it outputs
4. The judge: reliability report with the two bias checks
5. The system: score history from the changelog
6. Cost model

Postmortem: what went wrong and what you learned. A postmortem with no problems loses points.

Rules for every project: runs from a clean clone · has an eval · has a cost model · has a postmortem · you can explain every line.